Detailed Summary of the v5 ETH Report

Exact Recursive Tests, Certified Precision Savings,
Symmetry Speedups, Memory Costs, and Full-Algorithm Limits

Dr. Muralidhara Subbarao
Technical Report Prepared with the Assistance of AI Models ,
Dept. of Electrical and Computer Engineering,
Stony Brook University, New York,
murali.subbarao@stonybrook.edu,
https://www.integralresearch.net .

October 27, 2026

Abstract

This report summarizes revision 5 of the Ram Calculus investigation of OpenAI's manuscript *The Exponential Time Hypothesis is False*. V5 implements node-specific precision and certified symmetry inside an exact recursive tensor-contraction evaluator. Unlike the preceding local tests, the new experiment constructs and solves child networks; a nested fixture reaches two active levels and exercises non-scalar intermediate values and fixed creation-time lifts. All 300 fresh worker outputs on six fixtures match independent exact answers, and all 400 saved root local polynomials from active workers match independent polynomial oracles. The combined method reduces the nested median runtime from 472.726 to 245.233 milliseconds, a 1.93× speed ratio, while traced peak Python memory rises from 286.84 to 317.31 KiB. Its precision path falls from $5 \to 261 \to 389$ bits to $5 \to 23 \to 32$ bits. These successes are limited: asymmetric cases show little combined benefit and can incur symmetry overhead, and direct counting is much faster on every small fixture. Most importantly, the tests use an explicitly rescaled singleton-packet specialization, not the paper's literal active schedule or full SAT solver. We explain the proofs, exact checks, counterexamples, measurement protocol, and remaining work needed for a full-algorithm improvement. This is a summary of the recorded v5 evidence, not a new benchmark run, a new literature-status audit, or an independent verification of the source's headline claim.

Contents

1 Principal conclusions and the meaning of the comparison **4**

2 Why the literal source schedule was not executed **4**

2.1 The activation barrier 4

2.2 What replaces the source schedule in the tests 5

3 What is recursively computed **5**

3.1 The counting target and its deformation 5

3.2 How actual child calls arise 6

3.3 Exact extension arithmetic and creation-time lifts 6

4 The matched cases and variants **7**

4.1 Six fixtures, not 300 independent problems 7

4.2 The five measured variants 7

5 What the precision reductions prove **8**

5.1 Recovery certificates are recomputed for the node 8

5.2 The observed precision paths 9

5.3 Why old small-degree formulas fail in recursion 9

5.4 The stronger restricted-family result is not a timed result 9

6 How certified symmetry saves recursive work **10**

7 Validation and experimental protocol **10**

7.1 Independent exact checks 10

7.2 Counts of tests and measurements 10

7.3 What runtime and memory include 11

8 Measured speedups and important failures **11**

8.1 Complete timing table 11

8.2 Evidence of overhead and variability 11

8.3 The direct control is decisively faster at this scale 12

9 Memory costs and the amount of recursive work **12**

9.1 Traced peak allocation 12

9.2 Counters confirm that real work was removed 13

10 What is proved, demonstrated, or still proposed **13**

11 Limitations and the route to a full-algorithm improvement **14**

11.1 Coverage gaps 14
11.2 Necessary mathematical work 14
11.3 Useful implementation directions 15

12 The Ram Calculus contribution and its conventional interpretation 15

13 Evidence files and concise conclusion 15

1 Principal conclusions and the meaning of the comparison

The main advance in v5 [1] is that the precision and symmetry reductions are no longer tested only as isolated local formulas. They operate inside a recursive evaluator that creates child tensor networks, requests their values at certified precisions, and uses those values to reconstruct its own target. Correctness is checked more deeply than by comparing only a final scalar answer.

The conclusions fall into four different categories:

1. **Proved finite identities.** The report proves the specialized recursive contraction identity, valid node-specific recovery budgets, and orbit aggregation under explicit symmetry hypotheses. It also gives a concrete counterexample to using the earlier one-edge decoders outside their permitted class.

2. **Implemented exact behavior.** The evaluator executes active recursive calls, preserves the prescribed creation-time lifts, and agrees with independent count, polynomial, and diagnostic child-target oracles on the stated finite cases.

3. **Measured benefits and costs.** Combined precision and symmetry give approximately 1.83× and 1.93× median speed ratios on the symmetric-cycle and nested fixtures. The nested combined method uses about 10.6% more traced peak memory than the finite source comparator. Symmetry can be unprofitable on asymmetric inputs.

4. **Unproved extrapolations.** There is no demonstrated improvement to the paper's full SAT algorithm, no improved general worst-case exponent, and no practical advantage over direct counting on these tiny instances.

Throughout this summary, *source comparator* means an implementation of a **finite, rescaled specialization of the source's contraction identity**, using source-form Hermite recovery and a conservative precision schedule. It does not mean an author-supplied executable or a literal execution of the complete OpenAI algorithm. This qualification applies to every speed ratio in the report.

V5 retains the technical body, equation comparisons, and prior experimental records from revisions 1–4. The new recursive experiment is in v5 Section 1, physical PDF pages 6–16. Earlier v4 benchmarks are separate experiments and are not included in the new 300-worker total. Their speedups must not be multiplied by v5's ratios to claim a cumulative full-algorithm speedup.

2 Why the literal source schedule was not executed

2.1 The activation barrier

The pinned source's Appendix D uses

$$\lambda = \lceil \log_2 s \rceil, \qquad \varepsilon = \frac{1}{131072}, \qquad u_d = \left\lfloor \frac{\varepsilon \lambda}{2^d} \right\rfloor, \qquad a_d = 2 \sum_{j=0}^{d} u_j.$$

For the root parameter u_0 to be positive, it is necessary that

$$\lambda \geq 131072, \qquad s \geq 2^{131071} + 1.$$

The latter is a 39,457-digit integer. This is a barrier on the magnitude of the underlying instance-size parameter, not a claim that storing the integer s itself would require an astronomical amount of memory. The source also uses the leaf threshold

$$H = \left\lceil \frac{s}{\lfloor \sqrt{\lambda} \rfloor} \right\rceil .$$

A nonleaf must have $M > H$. At the smallest size satisfying the root positivity condition, $\lfloor \sqrt{\lambda} \rfloor = 362$, so activation also needs

$$M \geq \left\lceil \frac{2^{131071} + 1}{362} \right\rceil + 1.$$

Assigning an artificially large value of s to a four-edge network does not make it active: the same change raises H and keeps that network in the direct branch.

These conditions are necessary, not sufficient. The source's packet packing and related conditions impose additional requirements. In particular a literal active root has $a_0 \geq 2$, whereas the executable specialization sets $a = 1$. Large constants do not by themselves refute an asymptotic theorem. They do prevent interpreting the finite experiment as an implementation of that theorem's literal active regime.

2.2 What replaces the source schedule in the tests

The specialization sets $a = u = 1$, $L = 2$, and $\alpha = 4$ at every active node. It uses singleton real packets, variable grouping instead of the source's fixed 64 groups, manually specified small leaf thresholds, and strict child shrinkage instead of the full quantitative source shrinkage conditions. Its coefficient ring is a fixed four-dimensional extension sufficient for the degrees in the tests, rather than the source's size-dependent extension.

The surviving local identities are proved directly under these finite conditions. Their correctness does not depend on pretending that the large sufficient packing inequalities hold. Conversely, correctness of these local identities does not transfer the source's time and space theorems to the rescaled code.

The direct evaluator is therefore especially important: on these tiny inputs, direct summation is the closest faithful control for the source's actual small-instance behavior. The recursive comparator is intentionally forced to do additional work in order to test the proposed reductions.

3 What is recursively computed

3.1 The counting target and its deformation

Let M be the number of Boolean edge variables. Let $A(Y)$ and $B(X)$ be the products of the fixed tensors on the two sides of a bipartite network. The desired contraction is

$$Z = \sum_{X \in \{0,1\}^M} A(X)B(X).$$

For Boolean-valued constraint tensors this is an assignment count; for general weights it is a weighted contraction, not a SAT count.

For one real zero-slot edge per packet, the kernel is

$$W(T) = 1 + \frac{T}{2} - T^2 - \frac{T^3}{2}.$$

It satisfies $W(0) = 1$ and $W(-1) = W(1) = 0$. With $\ell_0(v) = 1 - v$ and $\ell_1(v) = v$, define

$$F(T) = \sum_{X,Y \in \{0,1\}^M} A(Y)B(X) \prod_{e=1}^{M} \ell_{Y_e}(W(T)X_e). \tag{1}$$

At $T = 0$, each factor enforces $X_e = Y_e$; hence $F(0) = Z$. The degree bound is $\deg F \le 3M$. The local polynomials at the two centers have integral kernel coefficients:

$$W(-1 + 4z) = 4z + 8z^2 - 32z^3,$$
$$W(1 + 4z) = -12z - 40z^2 - 32z^3.$$

3.2 How actual child calls arise

For a fixed local evaluation $w = W(t + 4\zeta)$,

$$\ell_y(wx) = \mathbf{1}_{\{y=0\}} + w(2y - 1)x.$$

Choosing one of the two terms for each edge produces a binary mode assignment $\beta \in \{0,1\}^M$. The corresponding unary packet tensors attached to old row and column ports are

	$\beta_e = 0$	$\beta_e = 1$
row port	$(1, 0)$	$(-w, w)$
column port	$(1, 1)$	$(0, 1)$.

After grouping old vertices, denote the resulting child contraction for group g by $C_g(\beta, w)$. Factoring the remaining assignment sums gives

$$F(t + 4\zeta) = \sum_{\beta \in \{0,1\}^M} \prod_{g} C_g(\beta, w). \tag{2}$$

The program creates these child networks and recursively evaluates them. It does not substitute their exhaustive scalar answers into a simulated recursion. Exhaustive calculations are reserved for the direct control, leaves, and independent validation.

Every child must have strictly fewer edges than its parent. Each parent edge contributes two child ports, so $\sum_g M_g = 2M$, even though each individual M_g is smaller than M. In the nested fixture the path is $4 \to 2 \to 1$: the root and the two-edge children are active; one-edge children are leaves. Counting the root as depth zero, the maximum active depth is one and the leaf depth is two. Thus "two active levels" is not the same as two interpolation centers or a long recursion-depth study.

3.3 Exact extension arithmetic and creation-time lifts

At working precision b, arithmetic takes place in

$$\mathcal{R}_b = (\mathbb{Z}/2^b\mathbb{Z})[U]/(U^4 + U + 1).$$

Reduction modulo two gives $\mathbb{F}_{16}$. At most 13 residue-distinct points are needed because $M \le 4$ and $\deg F \le 12$. Their differences are units, allowing exact interpolation over the lifted ring. Values retain all four coordinates, not just the scalar component.

A tensor entry also carries its creation precision p_c. Its canonical integer representative is fixed at creation and is carried unchanged into later, higher-precision calculations. For example, an entry created as $-1 \bmod 2^5$ is 31. Its prescribed lift at precision seven is still 31, not 127 obtained by re-evaluating the symbolic expression -1. The latter would replace the stored child input with a different one.

The nested fixture is meaningful because intermediate child values have nonzero U, U^2, or U^3 components. The source-comparator diagnostic has 66 nonbase active returns; the combined variant has 44. Their final root answers are scalar, but the computation does not falsely assume that all intermediate values must be scalar.

Correctness follows by induction on M: exact leaves imply exact child values, equation (2) supplies exact local evaluations, unit-separated interpolation recovers the local polynomials, and a certified target map recovers $F(0)$ to its requested precision.

4 The matched cases and variants

4.1 Six fixtures, not 300 independent problems

The four-edge cycle has row tensors on edges $(0,1)$ and $(2,3)$ and column tensors on $(0,2)$ and $(1,3)$. Each default truth table is $(0,1,1,1)$, with the first port as the least significant bit. The corresponding positive two-CNF has seven satisfying assignments.

Case	Description	Leaf H	Output bits	Answer
R01	Cycle, direct-branch control	4	5	7
R02	Cycle, active symmetric root	2	5	7
R03	First row replaced by $(0,0,1,1)$	2	5	5
R04	First row replaced by the zero table	2	5	0
R05	Independent unary constraints, two active levels	1	5	1
R06	Asymmetric positive 32-bit weighted cycle	2	129	See text

For R05, the four row unary tensors are $(1,0)$ and the four column unary tensors are $(1,1)$. Pairing vertices into groups makes four two-edge children. This is intentionally a simple, factorized input that exercises nested arithmetic; it is not a difficult SAT instance.

For R06, entry j of vertex i is $2^{31} + 17i + 3j + 1$, with $i, j \in \{0,1,2,3\}$. Its exact weighted result is

$$340282386569523140639734138112894692878.$$

The requested output modulus exceeds an elementary upper bound on the nonnegative answer, so the final modular result uniquely identifies the integer count or weight for every fixture.

4.2 The five measured variants

Direct Exhaustive contraction of the original small network.

S Finite source comparator: source-form Hermite weights, conservative precision increments, and all binary modes.

D The same finite recursive construction with a node-specific degree certificate and smaller certified precision increments.

G Source-comparator precision, but mode sums are replaced by certified symmetry-orbit sums.

DG Both node-specific degree-aware precision and symmetry.

All recursive variants use the same interpolation degree, exact ring arithmetic, and a common per-call least-recently-used cache holding at most 16 child values. Cache keys include the full immutable child network, creation precisions, edge count, and requested precision. Interpolation bases are also cached within the evaluator. These common engineering choices are included in measured cost, but they are not assertions about the source paper's memory-preserving implementation.

5 What the precision reductions prove

5.1 Recovery certificates are recomputed for the node

Write $c_{t,j} = [z^j]F(t + 4z)$, with $t \in \{-1, 1\}$. A rational Hermite target functional has the form

$$F(0) = \sum_{t,j} w_{t,j} c_{t,j}.$$

If this identity is valid on the node's polynomial class, define

$$e = \max\bigl(0, \max_{w_{t,j} \neq 0} [-v_2(w_{t,j})]\bigr).$$

Knowing every channel modulo 2^{p+e} suffices to determine the integral target modulo 2^p. Indeed, a channel error divisible by 2^{p+e}, multiplied by its weight, remains divisible by 2^p. The same reasoning applies coordinatewise to the extension ring. Odd denominator factors are inverted modulo the output modulus; powers of two are divided only after the accumulated numerator has the required divisibility.

For the finite two-center configuration, the source-form plan uses $q_- = q_+ = 3M$ channels per center and a conservative precision increment $64M$. Its actual compiled target weights have denominator loss $9M - 2$. The degree-aware plan instead chooses balanced prefixes satisfying $q_- + q_+ = 3M + 1$, sufficient for $\deg F \leq 3M$, and has loss $\lfloor 9M/2 \rfloor$.

M	$\deg F$ bound	S increment used	Exact loss of S weights	D increment used	D channel count
2	6	128	16	9	7
4	12	256	34	18	13
8	24	512	70	36	25

The $M = 8$ entry is a decoder unit test, not a recursive benchmark: the recursive engine supports only $M \leq 4$.

This table separates two sources of bit savings. Much of the reduction from $64M$ is the replacement of a conservative schedule by a sharp certificate for a chosen target map. Using the smaller degree-certified map reduces the loss further, for example from 34 to 18 at $M = 4$. There is no timed recursive ablation using only the exact loss of the unchanged S map. Therefore the measured S-to-D improvement should not be attributed entirely to lowering the number of retained channels.

5.2 The observed precision paths

For R02–R04, the requested root answer has five bits of precision. The working precisions are 261 bits for S/G and 23 for D/DG. For the 129-bit weighted answer in R06, they are 385 and 147 bits respectively. In the nested case R05:

$$\begin{aligned} \text{S and G:} \quad & 5 \longrightarrow 261 \longrightarrow 389, \\ \text{D and DG:} \quad & 5 \longrightarrow 23 \longrightarrow 32. \end{aligned}$$

The first increment is $64 \cdot 4 = 256$ versus 18; the next is $64 \cdot 2 = 128$ versus 9. The largest working width therefore drops from 389 to 32 bits, approximately a 91.8% reduction. This does *not* mean a 91.8% runtime or memory reduction: Python object overhead, interpolation, group discovery, and child-network construction remain.

Both plans still reconstruct all $3M + 1$ coefficients of each local polynomial using $3M + 1$ evaluations per center. Fewer target channels do not automatically mean fewer recursive evaluations. The measured D variant changes precision and target-map work, not the full interpolation evaluation count at a given executed node.

5.3 Why old small-degree formulas fail in recursion

The v4 degree-three decoder and one-W decoder were valid on their stated one-edge classes. They cannot be reused unchanged as the network grows. Two independent singleton packets with row and column unary tensors $(0, 1)$ produce

$$F(T) = W(T)^2, \qquad F(0) = 1.$$

Since W vanishes at both centers, the constant and linear local coefficients of W^2 vanish at both centers. Consequently both old formulas return zero, regardless of how many arithmetic bits are used. This is a realizable two-edge counterexample, not just an artificial polynomial with no network interpretation.

The correction is a *degree or family certificate at every node*, not simply a larger modulus. V5 checks its compiled weights exactly on a basis for the certified class and includes invalid-degree, invalid-family, and underprecision controls.

5.4 The stronger restricted-family result is not a timed result

Singleton packets imply $F(T) = G(W(T))$ with $\deg G \le M$. At the negative center, $W(-1 + 4z) = 4R(z)$, where $R(z) = z + 2z^2 - 8z^3$ has an integral formal inverse. The prefix through degree M recovers $G(1)$ with loss $2M$. The witness $2^{p-1}W^M$ shows that one fewer bit can conceal the target in all local coefficient observations for this unrestricted coefficient family.

The corresponding structural decoder is implemented and symbolically checked, but **it is not selected by any of S, D, G, or DG in the timed recursive tests**. The 32-bit nested maximum comes from the degree-aware 18- and 9-bit increments, not the structural $2M$ rule. The structural sharpness statement is not a lower bound for all Boolean-count problems or a theorem for general multi-edge packets.

6 How certified symmetry saves recursive work

Symmetry is accepted only if an edge permutation preserves the full finite computation: bipartite side, incidence and port ordering, tensor entries after axis permutation, creation precision, and the vertex partition up to a bijection of groups. Because singleton kernels are uniform, these checks preserve the complete deformation, not merely the answer at $T = 0$.

If Γ is the accepted group and O runs over its orbits of binary mode assignments, then

$$\sum_{\beta} \prod_{g} C_g(\beta, w) = \sum_{O} |O| \prod_{g} C_g(\beta_O, w).$$

This is multiplication by integer orbit sizes, not averaging by the group order; division by an even group order would be invalid in the modular ring. V5 checks disjointness and completeness of the orbit cover and recertifies symmetry at children using their actual stored tensors.

The symmetric four-edge cycle has four accepted permutations and seven mode orbits in place of sixteen mode assignments. Its child requests fall from 1664 to 728. The broken-symmetry and weighted fixtures have only the identity, so certification incurs overhead without reducing their mode sums. It would be incorrect to infer symmetry from the shape of the graph while ignoring changed tensor values.

The implementation enumerates all $M!$ permutations. That is acceptable for $M \leq 4$, but it is not an efficient general algorithm for finding useful symmetries. Also, fewer mode requests can change cache behavior and the number of actually executed child calls. The performance results include those interactions; they are not a universal ratio obtained by dividing 16 by 7.

7 Validation and experimental protocol

7.1 Independent exact checks

Each fixture has an independent exhaustive scalar count or weighted answer. A separate double assignment sum expands equation (1) in rational polynomial arithmetic and derives both complete root local polynomials. Every fresh worker's output is checked against those oracles.

The 200 active worker outputs contain 400 root local polynomials, two per output. A stricter post-run audit checks that neither center nor any required coefficient is missing. Diagnostic workers additionally compare every executed active child target with exhaustive ring-valued contraction. Thus agreement of the final scalar is not the only correctness evidence. Matched variants may create different integer representatives at different precisions, but each is checked at the required quotient precision under its own fixed-lift contract.

7.2 Counts of tests and measurements

The recorded suite [2] has six fixtures and five variants. For each pair it runs:

- Seven untraced timing workers: $6 \cdot 5 \cdot 7 = 210$.
- Two separate memory workers: $6 \cdot 5 \cdot 2 = 60$.
- One diagnostic worker: $6 \cdot 5 = 30$.

All 300 worker outputs passed their specified validation. Separately, 32 seeded random Boolean truth-table networks were checked with all four recursive variants, producing 128 method–instance checks. They are algebraic stress tests, not a benchmark distribution of hard CNFs. Precision unit tests include 167 monomial checks, 108 comparisons with the uncompiled source-form Hermite expression, 23 restricted-family basis checks, and 120 precision checks. Invalid grouping, nonshrinking children, bad creation precisions, and noncanonical stored values are rejected.

7.3 What runtime and memory include

The recorded environment is CPython 3.12.14 on Windows 11 build 26300, with processor identifier Intel64 Family 6 Model 140 Stepping 1. Workers are fresh processes, executed sequentially with rotated method order. Interpreter startup and imports are outside the runtime region; network construction, certificates, plans, interpolation, child creation, caching, exact arithmetic, and recovery are inside it. Independent validation occurs outside the timed and traced regions.

Memory figures are median peak allocations traced by Python's `tracemalloc`, in KiB. They do not comprehensively measure native allocations, interpreter overhead, operating-system working memory, or the source's theoretical space usage. Raw records also contain Windows process peak working sets, but those include startup and imports and are not treated as incremental algorithm memory. Tracing-run runtimes are never substituted for the untraced timing measurements.

8 Measured speedups and important failures

8.1 Complete timing table

	Median runtime, milliseconds					S/DG
Case	Direct	S	D	G	DG	ratio
R01	0.286	0.320	0.328	0.330	0.319	1.00
R02	0.272	51.920	45.371	38.368	28.331	1.83
R03	0.290	55.405	54.441	57.560	52.823	1.05
R04	0.285	54.865	50.988	43.091	44.003	1.25
R05	0.475	472.726	401.095	297.413	245.233	1.93
R06	0.286	62.502	56.971	76.165	60.481	1.03

Ratios use the unrounded medians in the saved records. R01 never enters the recursive branch, so its small differences are wrapper overhead and timing variability, not benefits of precision or symmetry reduction.

For R02, precision alone gives approximately 1.14×, symmetry alone 1.35×, and their combination 1.83×. For R05 the corresponding ratios are 1.18×, 1.59×, and 1.93×. The nested combined median is about 48.1% lower than S, not 93% lower: a speed ratio compares reciprocal runtimes and should not be confused with a percentage reduction in elapsed time.

8.2 Evidence of overhead and variability

There is no universal benefit from adding the symmetry machinery:

- In R03, no useful symmetry remains and G is about 3.9% slower than S. The combined S/DG ratio is only 1.05.
- In R06, G is about 21.9% slower than S; DG is about 6.2% slower than D. The overall S/DG ratio is only 1.03.
- In R04, DG is slightly slower than G, despite its smaller working precision. Bit savings do not force a runtime improvement.

Timing variability is material. R02 S ranges from 42.446 to 136.355 ms. R05 S ranges from 436.557 to 520.125 ms, while R05 DG ranges from 226.328 to 265.102 ms. The records include all repeats and their median absolute deviations. The small 1.03–1.05 ratios should not be described as statistically established, robust population gains. No such inference was performed.

8.3 The direct control is decisively faster at this scale

Across the five active fixtures, DG is still approximately 104–517 times slower than direct summation of the original tiny network. For R05, direct counting takes approximately 0.475 ms whereas DG takes 245.233 ms. The original answer space has only four Boolean edge variables, while the recursive specialization deliberately constructs many interpolation and child computations. These tests establish savings relative to that recursive comparator, not competitive small-instance SAT performance.

9 Memory costs and the amount of recursive work

9.1 Traced peak allocation

	Median peak traced Python allocation, KiB				
Case	Direct	S	D	G	DG
R01	8.20	8.51	8.51	8.51	8.51
R02	8.20	228.09	204.15	230.85	206.92
R03	8.20	226.19	203.10	228.02	204.93
R04	8.20	220.75	199.84	223.33	202.28
R05	14.12	286.84	246.86	356.92	317.31
R06	8.39	236.38	211.78	238.15	213.56

Precision alone lowers traced peak allocation on all active fixtures. The combined method uses roughly 8–10% less than S on the four single-active-level fixtures, but the nested case reverses that result:

- G uses about 24.4% more peak traced memory than S.
- DG uses about 10.6% more than S.
- DG uses about 28.5% more than D.

Thus the strongest nested runtime improvement trades additional memory for less work. The report attributes the overhead to group/orbit and provenance data together with temporary Python objects; it is not a component-by-component allocation proof or a bound for a lower-level, streaming

implementation. A much smaller integer width does not imply a proportionally smaller object-heavy program footprint.

9.2 Counters confirm that real work was removed

	Active calls		Child requests		Mode products	
Case	S	DG	S	DG	S	DG
R02	1	1	1664	728	416	182
R03	1	1	1664	1664	416	416
R04	1	1	1664	1040	416	260
R05	84	57	20256	11544	5064	2886
R06	1	1	1664	1664	416	416

Active calls are executed nonleaf calls, excluding cache hits; child requests include hits. Mode-product totals are accumulated over executed active calls. In R05, the change is from one root plus 83 active child calls to one root plus 56. Local evaluation totals fall from 1188 to 810. The diagnostic records also show nonzero crossing-mode products, ruling out a vacuous computation in which only the zero mode survives.

These counts are consistent with the measured benefit but should not be treated as independent runtime multipliers. Orbit selection, cache hits, precision-dependent arithmetic, and the number of executed children all interact. The precision-only variant retains the S call structure, and the combined variant retains the symmetry variant's corresponding structure on these fixtures.

10 What is proved, demonstrated, or still proposed

Claim	Evidence status	Boundary
Finite recursive identity and termination	Proved	Singleton packets, side-pure groups, strict child shrinkage, valid recovery and fixed lifts.
Node-specific precision losses	Proved and checked	Exact recovery for the specified finite target maps; not arbitrary full-source packet families.
Symmetry orbit aggregation	Proved and checked	Requires preservation of the full deformation, tensor values, port structure, creation data, and grouping.
Exact counts and local polynomials	Demonstrated	Six fixtures, 300 worker outputs, plus the separate random and unit tests; not general solver verification.
1.93× nested speed ratio	Measured	Median on one small, factorized nested fixture; not an asymptotic exponent change.
Nested maximum 389 to 32 bits	Proved budget and observed execution	The full local polynomial is still interpolated; the structural $2M$ plan was not timed.
Smaller memory in general	Not established	Nested DG uses 10.6% more traced peak memory than S.
Structural $2M$ loss	Proved restricted result, symbolically tested	For the singleton $G(W)$ family and its residue observations, not a universal SAT lower bound.

Claim	Evidence status	Boundary
Faster full OpenAI algorithm or better general SAT exponent	Not established	Needs full-source certificates, complete recurrence and bit-cost analysis, and compatible space guarantees.
Selective-channel evaluation and scalable adaptive symmetry	Proposed	Require new algorithms, correctness proofs, and implementation measurements.

11 Limitations and the route to a full-algorithm improvement

11.1 Coverage gaps

The finite benchmark does not exercise general multi-edge real packets, nonzero higher tile moments, unequal slot positions, large arrangements, the full source parameter schedule, or a broad recursion-depth range. All roots have only four edges. The one two-level example is easy and factorized, and R06 is weighted contraction rather than SAT. The random truth-table checks add algebraic variety but do not close those structural coverage gaps.

The ordinary direct branch, the rescaled recursive identity, and the full paper algorithm are therefore three different objects. Exactness of one does not independently validate every reduction and parameter interface of another. The experiment also does not verify the source's full Lean dependency closure or settle the external mathematical acceptance of its headline result.

11.2 Necessary mathematical work

The most consequential next steps are:

1. **Extend the local certificate.** Derive target maps and valid precision losses for the actual multi-packet polynomial families and nonzero higher modes of the source, preserving fixed creation-time lifts.
2. **Propagate bounds uniformly.** Prove that the improved losses hold for all recursively generated children, not just hand-selected root networks or favorable coefficient classes.
3. **Recompute the total recurrence.** Include interpolation, packet construction, mode enumeration, exact bit arithmetic, certificate construction, symmetry discovery, and recursion. A reduction in one local loss does not automatically reduce the dominant global exponent.
4. **Account for space.** Replace or bound the caches, stored bases, orbit catalogues, and provenance state consistently with the claimed space target. Less arithmetic can coexist with more workspace.
5. **Handle trivial symmetry.** A worst-case theorem cannot assume every network has useful automorphisms. Certification must have a controlled cost and a sound fallback when only the identity is present.

For example, a uniform reduction of a precision increment by a constant factor might improve constants while leaving its asymptotic order unchanged. Conversely, a family-aware observation scheme might ultimately reduce more than constants, but that requires a new global analysis; the present measurements do not provide one.

11.3 Useful implementation directions

Computing only the channels needed by the target functional, instead of fully interpolating both local polynomials, is a plausible direction. V5 has not implemented that algorithm; choosing a shorter list of weights does not itself eliminate the existing evaluations. Adaptive symmetry screening could avoid expensive group discovery on obviously asymmetric nodes, but it must preserve exact correctness and include its own cost.

Other useful next tests would include mixed-slot packets, genuinely nonzero higher-weight modes, deeper finite recursion, independently generated tensor families, and a separate comparator using the exact source-map loss without changing its channel selection. Streaming or low-level implementations should measure both Python-independent peak memory and arithmetic cost. These are proposed tests and extensions, not results already delivered by v5.

12 The Ram Calculus contribution and its conventional interpretation

The Ram Calculus viewpoint organizes the calculation around a declared measurement class, an exact target functional, an explicit precision certificate, and symmetries transported through the entire computation. That organization is useful here: it prevents extending a one-edge formula beyond its domain, makes fixed lifts part of the input contract, and gives auditable local changes rather than an unsupported global complexity claim.

The strongest new contributions within this report sequence are the recursive integration and its safeguards, the realizable W^2 counterexample, node-specific precision propagation, full-deformation symmetry tests, and the reproducible runtime/memory evidence. They improve the transparency of which reductions actually save work and which merely move cost elsewhere.

The underlying mechanisms are also standard Hermite interpolation, two-adic valuation error propagation, finite-field interpolation, tensor contraction, and finite-group orbit aggregation. A conventional method implementing the same formulas and certificates would obtain the same identities and savings. V5 does not establish that the benefit is unique to Ram Calculus, nor a historical priority claim for those underlying mathematical tools. The appropriate claim is a specified, tested integration informed by the Ram framework.

13 Evidence files and concise conclusion

This summary uses the finalized 66-page v5 report and the recorded benchmark evidence, without rerunning or changing those experiments. The important files in `eth-benchmark-v5/` are:

- `recursive_v5.py`: exact finite recursive evaluator.
- `precision_plans.py`: exact recovery plans and unit tests.
- `run_v5.py`: fresh-process benchmark harness.
- `results-20261027/results.json`: aggregate timings, memory, counts, counters, plans, platform information, and source hashes.
- `results-20261027/workers.jsonl`: the 300 raw worker records.

- `results-20261027/instances-and-oracles.json` and `selftest.json` in the same run directory: inputs, independent exact oracles, and self-test outcomes.
- `strict-saved-result-audit.json` and `final-verification.json`: completeness, preservation, and report integrity checks.

The original v5 source and PDF hashes were checked against their final manifest while preparing this summary. No new performance or source-status claim is implied by producing this document.

Bottom line. V5 successfully moves the precision and symmetry ideas into a genuine but deliberately small *rescaled recursive* computation. It obtains exact answers with much smaller working precision, and useful measured speedups on selected structured cases. The nested result is approximately 1.93× faster with 10.6% more traced peak memory, while the maximum working precision falls from 389 to 32 bits. The same evidence also exposes symmetry overhead, invalid decoder transfer, and the dominance of direct counting on tiny inputs. Literal active-schedule execution, scalable general certificates, full time/space analysis, and any stronger general SAT complexity result remain outside what has been established.

References

[1] M. Subbarao. *Ram Calculus and the Exponential Time Hypothesis: Exact Finite-Algebra Reformulations, Aggregated Boolean Jets, and Precision-Certified Counting. Revision 5: Exact Finite Recursive Tests, Precision Propagation, and Certified Symmetry.* October 27, 2026. Local report, `Ram-Calculus-Exponential-Time-Hypothesis-v5.tex` and corresponding 66-page PDF, especially Section 1, pp. 6–16. Retains the earlier technical body and experiments.

[2] M. Subbarao. *V5 finite recursive benchmark records.* October 27, 2026. `eth-benchmark-v5/results-20261027/`, with `strict-saved-result-audit.json` and `final-verification.json` in its parent directory. This summary reports the existing run; it does not supply an additional independent run.

[3] OpenAI. *The Exponential Time Hypothesis is False.* October 17, revised October 25, 2026. Source version examined by v5, commit `42907c9e999cc00d6abe7a50bfc1023250edd1b9`; Appendix D, `quantitative-contraction.tex`. https://github.com/openai/math/tree/42907c9e999cc00d6abe7a50bfc1023250edd1b9/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026. The citation identifies the analyzed version, not a fresh verification of repository status or of the manuscript's full theorem.

[4] M. Subbarao. *Ram Calculus and the Exponential Time Hypothesis*, revision 4. October 27, 2026. Local report, `Ram-Calculus-Exponential-Time-Hypothesis-v4.tex` and corresponding PDF. Prior one-edge precision and finite symmetry experiments; their measurements are distinct from the recursive v5 suite.